

Informática Gráfica y Visualización

Actividad 2 grupal: Proyecciones 3D

Componentes Grupo N01:

Belmonte Martín, Adrián

Blanco Wasmer, Pedro

Rodríguez Ales, Juan Antonio

Vega Castilla, Rubén

Moreno Pozuelo, Isabel

18 de mayo de 2026

Índice

1. Mundo Ciudad	3
2. Objetos	9
3. Control del teclado	32
4. Ejecución	32
5. Proyecciones	33

1. Mundo Ciudad

Para desarrollar nuestra actividad hemos creado una escena de una ciudad que contiene diferentes objetos: coche, carretera, carril bici, acerado, semáforo, paso de cebra, línea de stop, farolas, nubes, árbol, persona y bicicleta.

En la primera reunión concretamos qué es lo que queríamos desarrollar y poco a poco fuimos diseñando la escena. Se realizó un reparto de objetos para que cada uno de los participantes desarrollara una parte y así realizar una integración de todos los objetos en nuestra escena.

Se importan las librerías siguientes:

OpenGL.GL: es una especificación estándar que define una API multilenguaje y multiplataforma para escribir aplicaciones que produzcan gráficos 2D y 3D. La interfaz consiste en más de 250 funciones diferentes que pueden usarse para dibujar escenas tridimensionales complejas a partir de primitivas geométricas simples, tales como puntos, líneas y triángulos.

OpenGL.GLUT: es una biblioteca de utilidades para programar OpenGL que principalmente proporciona diversas funciones de entrada/salida con el sistema operativo. Entre las funciones que ofrece se incluyen declaración y manejo de ventanas y la interacción por medio de teclado y ratón. También posee rutinas para el dibujo de diversas primitivas geométricas que incluyen cubos, esferas, cilindros, entre otras.

OpenGL.GLU: es una colección de funciones de programación gráfica que proporciona funcionalidad adicional para las rutinas básicas de OpenGL.

Math: proporciona funciones para realizar operaciones matemáticas complejas que van más allá de las operaciones básicas como sumar, restar o multiplicar: cálculos trigonométricos, funciones exponenciales y logarítmicas, raíces y potencias...

Random: se utiliza para generar números pseudoaleatorios y realizar selecciones aleatorias en los programas.

Los ficheros que contienen nuestro desarrollo son los siguientes:

Act2.py:

Se definen los diferentes colores que se van a utilizar, por ejemplo: `dark_yellow_range = [yellow_3, yellow_4, yellow_5]`

A través del diccionario `visibilidad`, se controlan los objetos que se quieren mostrar en la escena, con los siguientes valores iniciales:

```
visibilidad = {  
    "ejes": False,  
    "coche": True,  
    "carril_bici": True,  
    "acerado": True,  
    "carretera": True,  
    "pasodecebra": True,  
    "semaforo": True,  
    "farola": True,  
    "nubes": True,  
    "bicicleta": True  
}
```

Con la función `gestiona_tecla`, el usuario puede seleccionar los objetos que quiere mostrar o eliminar de la escena.

```
def gestiona_tecla(key, x, y):  
    match key:  
        case b'\x1b' | b'q' | b'Q': # ESC, q ó Q  
            print(f"Tecla {key} pulsada -> Salir")  
            Salir ()
```

case b'0':

print("Tecla 0 pulsada -> Cambiar visibilidad de ejes")

cambiar_visibilidad("ejes")

case b'1':

print("Tecla 1 pulsada -> Cambiar visibilidad de coche")

cambiar_visibilidad("coche")

case b'2':

print("Tecla 2 pulsada -> Cambiar visibilidad de carril_bici")

cambiar_visibilidad("carril_bici")

case b'3':

print("Tecla 3 pulsada -> Cambiar visibilidad de acerado")

cambiar_visibilidad("acerado")

case b'4':

print("Tecla 4 pulsada -> Cambiar visibilidad de carretera")

cambiar_visibilidad("carretera")

case b'5':

print("Tecla 5 pulsada -> Cambiar visibilidad de pasodecebra")

cambiar_visibilidad("pasodecebra")

case b'6':

print("Tecla 6 pulsada -> Cambiar visibilidad de semaforo")

cambiar_visibilidad("semaforo")

case b'7':

print("Tecla 7 pulsada -> Cambiar visibilidad de farola")

cambiar_visibilidad("farola")

```

case b'8':

    print("Tecla 7 pulsada -> Cambiar visibilidad de nubes")

    cambiar_visibilidad("nubes")

case b'9':

    print("Tecla 7 pulsada -> Cambiar visibilidad de bicicleta y persona")

    cambiar_visibilidad("bicicleta")

case _:

    print(f"Tecla sin acción asignada: {key}")

```

La función `draw_viewport` configura el viewport y dibuja el mundo. Los diferentes viewport son: Viewport 1 (arriba-izquierda) proyección gabinete, vista por defecto (z-); Viewport 2 (arriba-derecha) proyección ortogonal vista posterior (z+); Viewport 3 (abajo-izquierda): proyección ortogonal, vista lateral derecha (x-); Viewport 4 (abajo-derecha): proyección en perspectiva simétrica.

Las funciones `draw_mundo_b()` y `draw_mundo` realizan las diversas llamadas a las funciones que definen los objetos que queremos que aparezcan en nuestra escena, por ejemplo: `igv_3dobjects.carretera()`

igv_3dobjects.py:

Contiene el desarrollo de los objetos que hemos creado para visualizar el Mundo Ciudad. Se explican en el apartado de objetos de este documento.

También aparecen las funciones de apoyo de esta asignatura: `solid_face_xz`, `solid_face_yz`, `solid_face_xy`, `empty_ortho`, `solid_ortho`, `density_face_xz`, `density_ortho`, `empty_face_xz`, `empty_pipe_y`, `empty_pipe_x`, `empty_face_yz`, `empty_pipe_x`, `empty_face_yz`

igv_basic3d.py:

Contiene las funciones de apoyo de la asignatura con diversos comentarios que explican su desarrollo:

```
def solid_face_xz(x_size, z_size, colors):
```

Función que dibuja una cara sólida en el plano XZ, formada por cubos de lado uno y colores aleatorios. La esquina inferior posterior izquierda de la cara es un cubo centrado en el punto

(0,0,0). El tamaño de la cara y la paleta de colores se definen a través de los parámetros de la función.

```
def solid_face_yz(y_size, z_size, colors):
```

Función que dibuja una cara sólida en el plano YZ, formada por cubos de lado uno y colores aleatorios.

Se construye a partir de la función `solid_face_xz(...)` girando 90 grados sobre el eje Z.

La esquina inferior posterior de la cara es un cubo centrado en el punto (0,0,0).

El tamaño de la cara y la paleta de colores se definen a través de los parámetros de la función.

```
def solid_face_xy(x_size, y_size, colors):
```

Función que dibuja una cara sólida en el plano XY, formada por cubos de lado uno y colores aleatorios.

Se construye a partir de la función `solid_face_xz(...)` girando -90 grados sobre el eje X.

La esquina inferior posterior izquierda de la cara es un cubo centrado en el punto (0,0,0).

El tamaño de la cara y la paleta de colores se definen a través de los parámetros de la función.

```
def empty_ortho(x_size, y_size, z_size, colors):
```

Función que dibuja un ortoedro hueco, formado por cubos de lado uno y colores aleatorios.

La esquina inferior posterior izquierda del ortoedro es un cubo centrado en el punto (0,0,0).

El tamaño del ortoedro y la paleta de colores se definen a través de los parámetros de la función.

```
def solid_ortho(x_size, y_size, z_size, colors):
```

Función que dibuja un ortoedro sólido, formado por cubos de lado uno y colores aleatorios.

La esquina inferior posterior izquierda del ortoedro es un cubo centrado en el punto (0,0,0).

El tamaño del ortoedro y la paleta de colores se definen a través de los parámetros de la función.

```
def empty_face_xz(x_size, z_size, colors):
```

Función que dibuja una cara hueca (sólo el contorno) en el plano XZ, formado por cubos de lado uno y colores aleatorios.

La esquina inferior posterior izquierda de la cara es un cubo centrado en el punto (0,0,0).

El tamaño de la cara y la paleta de colores se definen a través de los parámetros de la función.

igv_utils.py:

Módulo con funciones definidas para la asignatura:

`def print_matrix(tipo, mensaje):`

Muestra por consola el contenido de la matriz MODELVIEW o la matriz PROJECTION.

`def cube ():`

Dibuja un cubo centrado en el origen y de lado 1.

El nombre de las caras (frontal, posterior, etc.) corresponden a un punto de vista situado en el eje Z positivo, mirando hacia el eje Z negativo.

`def color_cube(colorv):`

Dibuja un cubo centrado en el origen y de lado 1 con todas las caras del mismo color

`def axes (xMin, xMax, yMin, yMax, zMin, zMax, tickmarks):`

Dibuja los 3 ejes de coordenadas con los siguientes colores:

Eje X: azul oscuro el semieje positivo y azul claro el semieje negativo.

Eje Y: verde oscuro el semieje positivo y verde claro el semieje negativo.

Eje Z: rojo oscuro el semieje positivo y rojo claro el semieje negativo.

`def draw_text(text, x, y):`

Muestra una cadena de texto en una posición concreta dentro de una escena 2D.

`def draw_text_3d (text, x, y, z):`

Muestra una cadena de texto en una posición concreta dentro de una escena 3D.

`def draw_label_viewport(text, vp_w, vp_h):`

Dibuja una etiqueta 2D en la esquina superior izquierda del viewport actual.

Permite mostrar varias líneas si el texto contiene saltos de línea.

2. Objetos

Coche: El coche se ha desarrollado con `solid_ortho` y `solid_face`, según las piezas que lo componen: chasis, ventanas, cabina, ruedas y luces.



```
def coche2():
```

```
    #Colores
```

```
    c_chasis = dark_blue_range
```

```
    # c_cabina = [0.6, 0.8, 1.0]
```

```
    # c_rueda = [0.1, 0.1, 0.1]
```

```
    # c_luz_del = [1.0, 0.0, 0.0]
```

```
    # c_luz_tras = [1.0, 0.0, 0.0]
```

```
    #Cubo Inferior (Chasis)
```

```
    # Dimensiones: Largo=20, Alto=5, Ancho=6
```

```
    glPushMatrix()
```

```
    glTranslatef(0, 2, 0) # Base en Y=2 -> El techo queda en Y=7 (2 + 5)
```

```
    solid_ortho(20, 5, 6, c_chasis)
```

```
    glPopMatrix()
```

```
    # cubo Superior
```

```
    # Dimensiones para no sobresalir: Largo=16, Alto=4, Ancho=4
```

```
    # Centrado en X: (20 - 16) / 2 = 2
```

```
    # Centrado en Z: (6 - 4) / 2 = 1
```

```

glPushMatrix()
glTranslatef(2, 7, 1) # Y=7 apila el cubo justo en el techo del chasis
solid_ortho(16, 4, 4, dark_red_range)
glPopMatrix()

# Ruedas

# Tamaño: Largo=3, Alto=3, Ancho=2

# Chasis en Z va de 0 a 6.

# Izquierda: z=0 (empieza en el borde y ocupa hasta z=2)

# Derecha: z=4 (empieza en 4 y ocupa hasta z=6)

rueda_x, rueda_y, rueda_z = 3, 3, 2
posiciones_ruedas = [
    (3, 0, 0), # Delantera Izquierda
    (3, 0, 4), # Delantera Derecha
    (14, 0, 0), # Trasera Izquierda
    (14, 0, 4) # Trasera Derecha
]

for (rx, ry, rz) in posiciones_ruedas:
    glPushMatrix()
    glTranslatef(rx, ry, rz)
    solid_ortho(rueda_x, rueda_y, rueda_z, dark_red_range)
    glPopMatrix()

# Luces Delanteras
posiciones_luces_del = [
    (0, 4, 0.5),
    (0, 4, 4.5)
]

```

```

]
for (lx, ly, lz) in posiciones_luces_del:
    glPushMatrix()
    glTranslatef(lx, ly, lz)
    solid_ortho(1, 2, 1, dark_yellow_range)
    glPopMatrix()

# Luces Traseras
posiciones_luces_tras = [
    (19, 4, 0.5),
    (19, 4, 4.5)
]
for (lx, ly, lz) in posiciones_luces_tras:
    glPushMatrix()
    glTranslatef(lx, ly, lz)
    solid_ortho(1, 2, 1, dark_red_range)
    glPopMatrix()

# # Lateral Derecho (Cara exterior en Z = 5)
# glPushMatrix()
# glTranslatef(4, 8, 5.05) # 5.05 sobresale ligeramente de la pared derecha
# solid_face_xy(12, 2, light_grey_range)
# glPopMatrix()

# Lateral Derecho
# Dimensiones: Largo=10, Alto=2
glPushMatrix()
# Z=4.05 "saca" la ventana del interior del chasis y la pega a la superficie exterior derecha
glTranslatef(5, 8, 4.05)

```

```
solid_face_xy(10, 2, light_grey_range)
```

```
glPopMatrix()
```

```
#Lateral Izquierdo (Cara exterior en Z = 1)
```

```
glPushMatrix()
```

```
glTranslatef(4, 8, 0.95) #
```

```
solid_face_xy(12, 2, light_grey_range)
```

```
glPopMatrix()
```

```
#Parabrisas Trasero (Parte trasera de la cabina, X = 18)
```

```
glPushMatrix()
```

```
# Se sitúa justo en la pared trasera (X = 18.05)
```

```
glTranslatef(18.05, 8, 2)
```

```
solid_face_yz(2, 2, light_grey_range)
```

```
glPopMatrix()
```

```
#Parabrisas Delantero
```

```
# Dimensiones: Alto (Y) = 2, Ancho (Z) = 2
```

```
glPushMatrix()
```

```
# X=2.05 coloca el plano justo en la superficie frontal exterior
```

```
# Y=8 centra la ventana verticalmente en la cabina
```

```
# Z=3.95 compensa el avance de la función para que quede centrado de Z=2 a Z=4
```

```
glTranslatef(2.05, 8, 3.95)
```

```
solid_face_yz(2, 2, light_grey_range)
```

```
glPopMatrix()
```

Carril bici: El carril bici se ha desarrollado con `solid_ortho` y color `light_green_range`.



```
def carril_bici():  
    glPushMatrix()  
    solid_ortho(200, 1, 15, light_green_range)  
    glPopMatrix()
```

Acerado: El acerado se ha implementado con `solid_ortho` y color `light_grey_range`. Contiene un bordillo junto a la carretera y unas baldosas amarillas.



```
def acerado ():  
    glPushMatrix()  
    # Base principal de la acera  
    solid_ortho(200, 1, 40, light_grey_range)  
  
    # Bordillo junto al carril bici  
    glPushMatrix()  
    glTranslatef(0, 1, 38)  
    solid_ortho(200, 1, 2, dark_grey_range)
```

```

glPopMatrix()

# Bordillo junto a la carretera

glPushMatrix()

glTranslatef(0, 1, 0)

solid_ortho(200, 1, 2, dark_grey_range)

glPopMatrix()

# Franja diferenciadora tipo baldosa táctil

x = 0

while x < 200:

    glPushMatrix()

    glTranslatef(x, 1, 3)

    solid_ortho(6, 1, 2, dark_yellow_range)

    glPopMatrix()

    x += 10

glPopMatrix()

```

Carretera: La carretera de ha implementado con solid_ortho y color dark_grey_range. Se han incluido unas líneas discontinuas blancas.



```
def carretera ():  
    glPushMatrix()  
    solid_ortho(200, 1, 40, dark_grey_range)  
    # Línea discontinua central (en z ~ mitad de 40)  
    dash_len = 6  
    gap = 6  
    z_line = 19 # centro aprox (0..39)  
    x = 0  
    while x < 200:  
        glPushMatrix()  
        glTranslatef(x, 1, z_line)      # y=1 para quedar encima del asfalto  
        solid_ortho(dash_len, 1, 2, [grey_1]) # bloque blanco  
        glPopMatrix()  
        x += dash_len + gap  
    glPopMatrix()
```

Paso de cebra y Línea stop: A través de la función `solid_ortho` se ha implementado el paso de cebra y la línea stop.



```
def pasodecebra():  
    glPushMatrix()  
    largo_x= 20  
    ancho_z= 2  
    franjas= 10  
    separacion= 2  
    i=0  
  
    for i in range(franjas):  
        glPushMatrix()  
  
        # Desplazar cada franja en Z  
        glTranslatef(0, 0, i * (ancho_z + separacion))  
  
        # Dibujar franja blanca
```



```

solid_ortho(
    int(largo_x), # tamaño en X
    2,           # grosor en Y
    int(ancho_z),# tamaño en Z
    [grey_1]

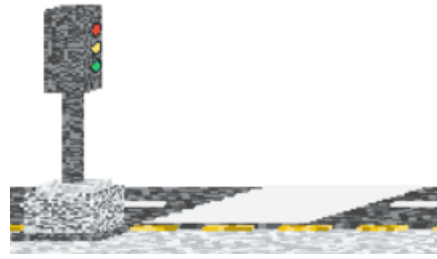
)

glPopMatrix()
glPopMatrix()

def linea_stop():
    glPushMatrix()
    largo_x= 8
    ancho_z= 15
    solid_ortho(
        int(largo_x), # tamaño en X
        2,           # grosor en Y
        int(ancho_z), # tamaño en Z
        [grey_1])
    glPopMatrix()

```

Semáforo: Para implementar el semáforo se ha utilizado la función `empty_ortho` para crear la base, el poste y la pantalla. Las luces se han generado a través de la función `circulo_luz` y los colores: `dark_red_range`, `yellow_dark_range` y `green_dark_range`.



```
def semaforo():
```

```
    escala = 0.5 # MODIFICAR AQUI PARA REDUCIR O AUMENTAR
```

```
    base_w = 18
```

```
    base_h = 20
```

```
    base_d = 18
```

```
    palo_w = 4
```

```
    palo_h = 50
```

```
    palo_d = 4
```

```
    pantalla_w = 12
```

```
    pantalla_h = 42
```

```
    pantalla_d = 10
```

```
def base():
```

```
    glPushMatrix()
```

```
    empty_ortho(base_w, base_h, base_d, grey_range)
```

```
    # Zócalo inferior más ancho para mayor estabilidad visual
```

```
    glPushMatrix()
```

```
    glTranslatef(-3, -4, -3)
```

```
    empty_ortho(base_w + 6, 4, base_d + 6, dark_grey_range)
```

```
    glPopMatrix()
```

```
    # Pedestal superior para rematar la transición al poste
```

```
    glPushMatrix()
```

```
    glTranslatef(2, base_h, 2)
```

```

empty_ortho(base_w - 4, 4, base_d - 4, light_grey_range)
glPopMatrix()
glPopMatrix()

```

```

def palo():
    glPushMatrix()
    glTranslate(base_w / 2 - palo_w / 2, base_h, base_d / 2 - palo_d / 2)
    empty_ortho(palo_w, palo_h, palo_d, dark_grey_range)
    glPopMatrix()

```

```

def pantalla():
    pantalla_x = base_w / 2 - pantalla_w / 2
    pantalla_y = base_h + palo_h
    pantalla_z = base_d / 2 - pantalla_d / 2

```

```

def circulo_luz(cx, cy, cz, radio, color):
    glPushMatrix()
    glColor3f(color[0], color[1], color[2])
    glBegin(GL_POLYGON)
    segmentos = 36
    for i in range(segmentos):
        angulo = 2 * pi * i / segmentos
        glVertex3f(cx + radio * cos(angulo), cy + radio * sin(angulo), cz)
    glEnd()
    glPopMatrix()

```

```

def centros_verticales(radio, separacion, cantidad, alto_total):
    alto_grupo = cantidad * (2 * radio) + (cantidad - 1) * separacion
    y_inicio = (alto_total - alto_grupo) / 2 + radio
    return [y_inicio + i * (2 * radio + separacion) for i in range(cantidad)]

```

```

glPushMatrix()
glTranslatef(pantalla_x, pantalla_y, pantalla_z)
empty_ortho(pantalla_w, pantalla_h, pantalla_d, dark_grey_range)
glPopMatrix()

```

```

def dibujar_luz(indice, color_rango):
    glPushMatrix()
    glTranslatef(pantalla_x, pantalla_y, pantalla_z)
    radio = 2.4
    separacion = 4.5

```

```

y = centros_verticales(radio, separacion, 3, pantalla_h)
x_centro = pantalla_w / 2
z_frente = pantalla_d + 0.2
circulo_luz(x_centro, y[indice], z_frente, radio + 0.9, black_5)
circulo_luz(x_centro, y[indice], z_frente + 0.01, radio, choice(color_rango))
glPopMatrix()

def luz_arriba():
    dibujar_luz(2, dark_red_range)

def luz_medio():
    dibujar_luz(1, dark_yellow_range)

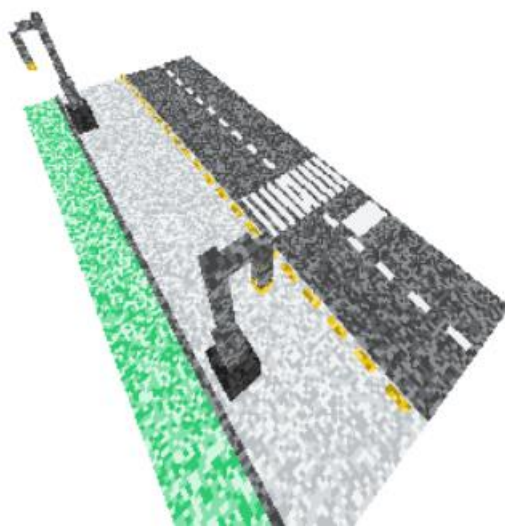
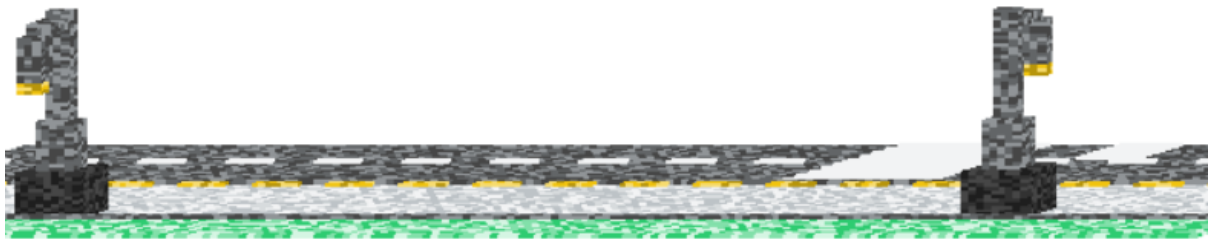
def luz_abajo():
    dibujar_luz(0, dark_green_range)

luz_arriba()
luz_medio()
luz_abajo()

glPushMatrix()
glScalef(escala, escala, escala)
base()
palo()
pantalla()
glPopMatrix()

```

Farolas: Para desarrollar las farolas se han utilizado las funciones `empty_pipe_y` y `solid_face_xz`. Contiene `base1`, `base2`, Barra vertical central, Brazo horizontal derecho y farol colgante. Aparecen dos farolas en la escena.



```
def farolav4():
    # Base 1
    glPushMatrix()
    empty_pipe_y(9, 12, 9, very_dark_grey)
    glTranslatef(0, 12, 0)
    solid_face_xz(9, 9, very_dark_grey)
    glPopMatrix()

    # Base 2
    glPushMatrix()
    glTranslatef(2, 13, 2)
```

```
empty_pipe_y(5, 12, 5, dark_grey_range)
glTranslatef(0, 12, 0)
solid_face_xz(5, 5, dark_grey_range)
glPopMatrix()
```

```
# Barra vertical central (Sostiene toda la estructura)
```

```
glPushMatrix()
glTranslatef(3, 25, 3)
empty_pipe_y(3, 30, 3, dark_grey_range)
glPopMatrix()
```

```
# Brazo horizontal derecho (Se mantiene arriba, en Y = 50)
```

```
glPushMatrix()
glTranslatef(6, 50, 3)
empty_pipe_x(6, 3, 3, dark_grey_range)
glPopMatrix()
```

```
# Farol colgante (Mirando hacia abajo)
```

```
# Se posiciona en el extremo del brazo horizontal (X = 12)
```

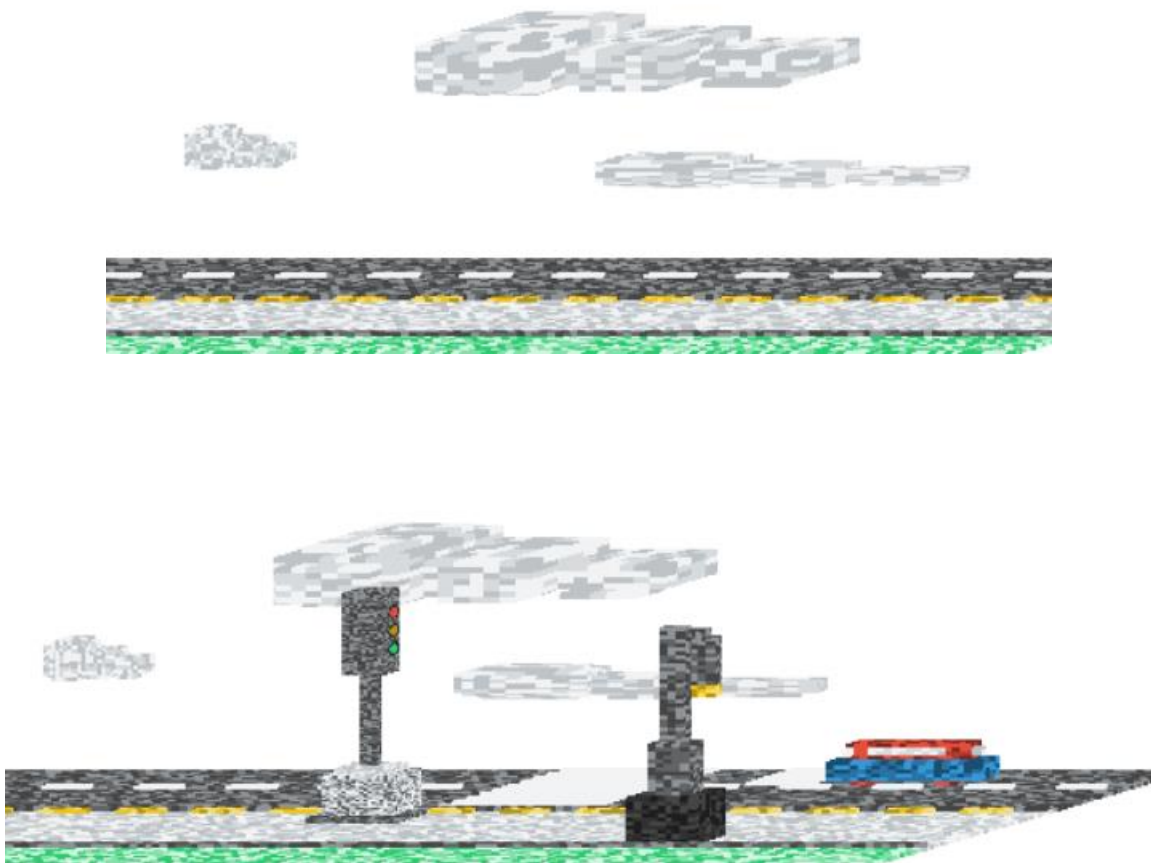
```
# Se desplaza hacia abajo en el eje Y (Y = 38) para que cuelgue de la barra
```

```
glPushMatrix()
glTranslatef(12, 38, 3)
empty_pipe_y(3, 12, 3, dark_grey_range) # El farol ahora se extiende hacia abajo
glPopMatrix()
glPushMatrix()
# Se posiciona en Y=35 (3 unidades por debajo de la boca del farol)
glTranslatef(12, 35, 3)
```

```
#
```

```
empty_pipe_y(3, 12, 3, dark_yellow_range)
glPopMatrix()
```

Nubes: Las nubes se han implementado con la función `empty_ortho` y `solid_face` junto con varios rebordes para darle su apariencia.



```

def nube (color = grey_range):
    def bloque_grande(g_color):
        # Cuerpo principal
        empty_ortho(8, 8, 8, g_color)

        # Reborde frontal
        glPushMatrix()
        glTranslatef(-1, 1, 0)
        solid_face_yz(6, 6, g_color)
        glPopMatrix()

        # Rebordes laterales
        glPushMatrix()
        glTranslatef(1, 1, -1)
        solid_face_xy(6, 6, g_color)
        glTranslatef(0, 0, 9)
        solid_face_xy(6, 6, g_color)
        glPopMatrix()

        # Rebordes superior e inferior
        glPushMatrix()
        glTranslatef(1, 8, 1)
        solid_face_xz(6, 6, g_color)
        glTranslatef(0, -9, 0)
        solid_face_xz(6, 6, g_color)
        glPopMatrix()

```



```

# Reborde inferior

# glPushMatrix()

# glTranslatef(1, -1, 1)

# solid_face_xz(6, 6, g_color)

# glPopMatrix()


def bloque_medio(m_color):
    empty_ortho(7, 6, 6, m_color)


# Reborde inferior

glPushMatrix()

glTranslatef(1, -1, 2)

solid_face_xz(5, 4, m_color)

glPopMatrix()


def bloque_pequeno(p_color):
    empty_ortho(7, 4, 4, p_color)


# Reborde inferior

glPushMatrix()

glTranslatef(1, -1, 3)

solid_face_xz(5, 2, p_color)

glPopMatrix()


glPushMatrix()

glTranslatef(1, 1, 0)

```

```

bloque_grande(color)
glTranslatef(8, 0, 0)
bloque_medio(color)
glTranslatef(7, 0, 0)
bloque_pequeno(color)
glPopMatrix()

```

Árbol: El árbol se ha implementado con la función `empty_ortho` y `density_ortho` para crear el tronco y la copa.

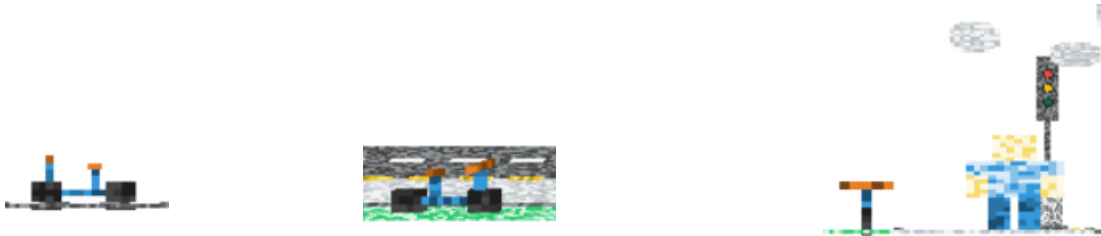


```

def tree(color_trunk, color_top):
    glMatrixMode(GL_MODELVIEW)
    glPushMatrix()
    empty_ortho(10, 50, 10, color_trunk)
    glPopMatrix()
    glPushMatrix()
    glTranslatef(-15,35,-15)
    density_ortho(40, 40, 40, 5, color_top)
    glPopMatrix()

```

Bicicleta: La bicicleta se ha creado a través de la función `solid_ortho`. Las partes que componen este objeto son: sillín, manillar, barras y ruedas.



`def bicicleta ():`

`#colores`

`color_ruedas = very_dark_grey`

`color_cuadro = dark_blue_range`

`color_asiento = dark_brown_range`

`# rueda trasera (Posicionada atrás en Z=0)`

`# Ancho X=1, Alto Y=4, Profundidad Z=4`

`glPushMatrix()`

`solid_ortho(1, 4, 4, color_ruedas)`

`glPopMatrix()`

`# rueda delantera (Posicionada adelante en Z=10)`

`# Ancho X=1, Alto Y=4, Profundidad Z=4`

`glPushMatrix()`

`glTranslatef(0.0, 0.0, 10.0) # Desplazada hacia adelante en el eje Z`

`solid_ortho(1, 4, 4, color_ruedas)`

`glPopMatrix()`

`# Conectar ambas ruedas`

`# Barra inferior central (une los ejes de las ruedas a la altura Y=2)`

`glPushMatrix()`

```

glTranslatef(0.0, 2.0, 3.0) # Centrada en X, elevada en Y, empieza tras la rueda trasera
solid_ortho(1, 1, 8, color_cuadro) # Barra horizontal de largo Z=8
glPopMatrix()

# Barra del sillín sube en Y=2 desde el centro Z=6
glPushMatrix()
glTranslatef(0.0, 3.0, 5.0)
solid_ortho(1, 3, 1, color_cuadro) #3 unidades de alto
glPopMatrix()


# barra del manillar (Horquilla delantera en Z=11, sube hasta Y=7)
glPushMatrix()
glTranslatef(0.0, 3.0, 11.0)
solid_ortho(1, 4, 1, color_cuadro) # Sube 4 unidades de alto
glPopMatrix()


# Sillín (En la punta de la barra del sillín Y=6, Z=5)
glPushMatrix()
glTranslatef(0.0, 6.0, 4.5)
solid_ortho(1, 1, 2, color_asiento) # Un asiento pequeño alargado
glPopMatrix()


# Manillar
glPushMatrix()
glTranslatef(-2.0, 7.0, 11.0) # Se mueve a la izquierda para centrar la barra transversal
solid_ortho(5, 1, 1, color_asiento) # Barra transversal de 5 de ancho en X
glPopMatrix()

```

Persona: Para implementar la persona se ha utilizado la función `solid_ortho`. Se han creado las siguientes partes: cabeza, camiseta, brazos derecho e izquierdo, pantalón y zapatos.



```
def persona ():
```

```
    color_piel = light_yellow_range      # Para cabeza y manos
```

```
    color_camiseta = light_blue_range    # Para cuerpo
```

```
    color_pantalón = dark_blue_range # Para las piernas
```

```
    color_zapatos = very_dark_grey # Para la base de los pies
```

```
    color_ojos = dark_blue_range
```

```
    color_boca = very_dark_grey
```

```
    glPushMatrix()
```

```
    # pierna izquierda (X = 0, Y = 0, Z = 0) por eso no lleva ningún glTranslatef
```

```
    #
```

```
    glPushMatrix()
```

```
    # Zapatos (Alto 1)
```

```
    solid_ortho(3, 1, 3, color_zapatos)
```

```
    # Pantalón ( Y=1 ( altura zapata), alto 7 del pantalón)
```

```
    glTranslatef(0.0, 1.0, 0.0)
```

```

solid_ortho(3, 7, 3, color_pantalon)
glPopMatrix()
#pantalon derecha me voy al 4,0,0
# X=4 Ancho pantalón 3 + 1 de separación por eso son 4 de separar
glPushMatrix()
glTranslatef(4.0, 0.0, 0.0)
# Zapatos
solid_ortho(3, 1, 3, color_zapatos)
# Pantalón
glTranslatef(0.0, 1.0, 0.0)
solid_ortho(3, 7, 3, color_pantalon)
glPopMatrix()
# cuerpo sobre las piernas
glPushMatrix()
glTranslatef(0.0, 8.0, 0.0) # 7 de pantalón +1 de zapato
solid_ortho(7, 8, 3, color_camiseta)
glPopMatrix()

# brazo izquierdo

# Al lado izquierdo del torso. tendrá Ancho=3, Alto=8 (3 de manga de camisetas y 5 de
brazo (piel))
glPushMatrix()
glTranslatef(-3.0, 8.0, 0.0) # me desplazo a -3 para poner añadir el brazo izquierdo,
# manga camiseta
glTranslatef(0.0, 5.0, 0.0)
solid_ortho(3, 3, 3, color_camiseta)
# brazo (piel )
glTranslatef(0.0, -5.0, 0.0) # (Alto 5) (pero hacia abajo)

```

```

solid_ortho(3, 5, 3, color_piel)

glPopMatrix()

#brazo derecho

#situado a la derecha del cuerpo que termina en x=7. tendrá Ancho=3, Alto=8 (3 de
manga de camisetas y 5 de brazo (piel))

glPushMatrix()

glTranslatef(7.0, 8.0, 0.0)

# manga

glTranslatef(0.0, 5.0, 0.0)

solid_ortho(3, 3, 3, color_camiseta)

# brazo piedl (Alto 5) (pero hacia abajo)

glTranslatef(0.0, -5.0, 0.0)

solid_ortho(3, 5, 3, color_piel)

glPopMatrix()


# (Centrada en X = 0.5, Y = 16, Z = -1.5)

# Cabeza 6x6x6. Centrado respecto al cuerpo que mide 7.

glPushMatrix()

# Y=16 total altura desde zapatos hasta vin de cuerpo. Se mueve 0.5 en X para centrar la
cabeza de 6 sobre el de 7

glTranslatef(0.5, 16.0, -1.5)

solid_ortho(6, 6, 6, color_piel)

glPopMatrix()

```

3. Control del teclado:

A través del teclado se pueden añadir objetos o quitar según la leyenda que aparece de la siguiente manera:

```
Perspectiva simétrica

Teclas:
0 - Ejes
1 - Coche
2 - Carril bici
3 - Acerado
4 - Carretera
5 - Paso de cebra
6 - Semáforo
7 - Farola
8 - Nube
9 - Bicicleta y persona
ESC/Q/q - Salir
```

4. Ejecución:

Al ejecutar Act2.py se visualiza nuestro Mundo Ciudad. El usuario, a través del teclado, puede mostrar o quitar los objetos que hemos creado, y así poco a poco, se visualizan en las distintas proyecciones todos los objetos.

```
***
Tecla 2 pulsada -> Cambiar visibilidad de carril_bici
Cambio visibilidad de carril_bici a visible
(0) ejes: True. (1) coche: True. (2) carril_bici: True. (3) acerado: False. (4) carretera
: False. (5) pasodecebra: False. (6) semaforo: True. (7) farola: False. (8) nubes: False.
```


5. Proyecciones:

El resultado final con todos los objetos de nuestro Mundo Ciudad es el que se muestra en la imagen.

Se aprecian las distintas proyecciones: paralela oblicua caballera o gabinete, paralela ortogonal vista frontal o posterior, paralela ortogonal vista lateral derecha o izquierda y perspectiva simétrica u oblicua.

